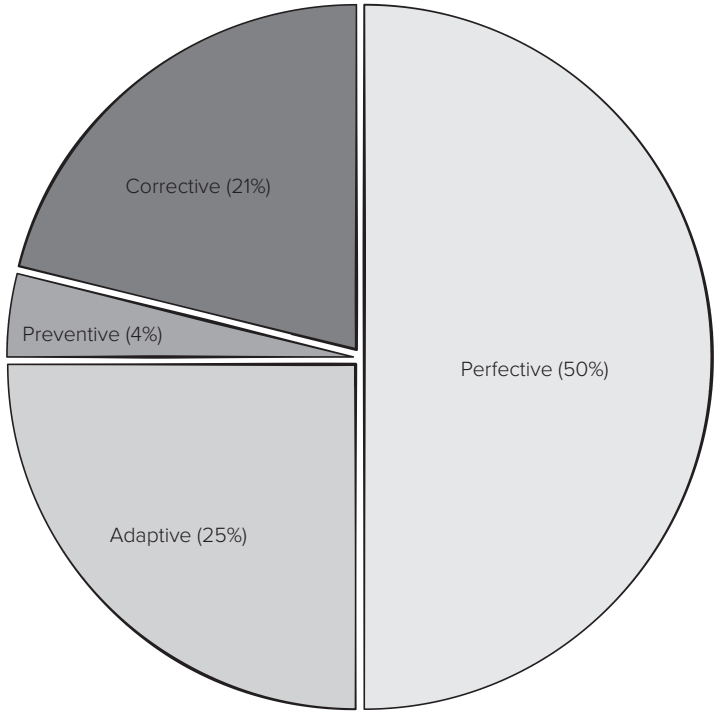
discovered after the software has been delivered to a customer's end user. *Adaptive maintenance* is reactive modification of software after delivery to keep the software usable in a changing end-user environment. *Perfective maintenance* is the proactive modification of the software after delivery to provide new user features, better program code structure, or improved documentation. *Preventive maintenance* is the proactive modification of the software after delivery to detect and correct product faults before they are discovered by users in the field [SWEBOK[4]]. Proactive maintenance can be scheduled and planned for. Reactive maintenance is often described as *firefighting* because it cannot be planned for and must be attended immediately for software systems that are critical to the success of the end-users' activities. Figure 4.5 shows that only 21 percent of the developers' time is typically spent on corrective maintenance.

For an agile evolutionary process model like the one described in this chapter, developers release working partial solutions with the creation of each incremental prototype. Much of the engineering work done is preventive or perfective maintenance as new features are added to the evolving software system. It is tempting to think that maintenance is handled simply by planning to make another trip around the spiral. But software problems cannot always be anticipated, so repairs may need to be made quickly and developers may be tempted to cut corners when trying to repair the broken

**FIGURE 4.5**

**Distribution of maintenance effort**




4 SWEBOK refers to the Software Engineering Body of Knowledge, which can be accessed using the following link: https://www.computer.org/web/swebok/v3.

software. Developers may not want to spend time on risk assessment or planning. Yet, developers cannot afford to make changes to software without considering the possibility that the changes required to fix one problem will cause new problems to other portions of the program.

It is important to understand the program code before making changes to it. If developers have documented the code, it will be easier to understand if other people need to do the maintenance work. If the software is designed to be extended, maintenance may also be accomplished more easily, other than emergency defect repairs. It is essential to test the modified software carefully to ensure software changes have their intended effect and do not break the software in other places.

The task of creating software products that are easy to support and easy to maintain requires careful and thoughtful engineering. A more detailed discussion on the task of maintaining and supporting software after delivery is presented in Chapter 27.

## Recommended Software Process Steps

1. Requirements engineering
   - Gather user stories from all stakeholders.
   - Have stakeholders describe acceptance criteria user stories.
2. Preliminary architectural design
   - Make use of paper prototypes and models.
   - Assess alternatives using nonfunctional requirements.
   - Document architecture design decisions.
3. Estimate required project resources
   - Use historic data to estimate time to complete each user story.
   - Organize the user stories into sprints.
   - Determine the number of sprints needed to complete the product.
   - Revise the time estimates as use stories are added or deleted.
4. Construct first prototype
   - Select subset of user stories most important to stakeholders.
   - Create paper prototype as part of the design process.
   - Design a user interface prototype with inputs and outputs.
   - Engineer the algorithms needed for first prototypes.
   - Prototype with deployment in mind.
5. Evaluate prototype
   - Create test cases while prototype is being designed.
   - Test prototype using appropriate users.
   - Capture stakeholder feedback for use in revision process.
6. Go, no-go decision
   - Determine the quality of the current prototype.
   - Revise time and cost estimates for completing development.
   - Determine the risk of failing to meet stakeholder expectations.
   - Get commitment to continue development.
7. Evolve system
   - Define new prototype scope.
   - Construct new prototype.
   - Evaluate new prototype and include regression testing.
   - Assess risks associated with continuing evolution.
8. Release prototype
   - Perform acceptance testing.
   - Document defects identified.
   - Share quality risks with management.
9. Maintain software
   - Understand code before making changes.
   - Test software after making changes.
   - Document changes.
   - Communicate known defects and risks to all stakeholders.

## 4.10 Summary

Every project is unique, and every development team is made up of unique individuals. Every software project needs a road map, and the process of developing software requires a predictable set of basic tasks (communication, planning, modeling, construction, and deployment). However, these tasks should not be performed in isolation and may need to be adapted to meet the needs of each new project. In this chapter, we suggested the use of a highly interactive, incremental prototyping process. We think this is better than producing rigid product plans and large documents prior to doing any programming. Requirements change. Stakeholder input and feedback should occur early and often in the development process to ensure the delivery of useful product.

We suggest the use of an evolutionary process model that emphasizes frequent stakeholder involvement in the creation and evaluation of incremental software prototypes. Limiting requirements engineering artifacts to the set of minimal useful documents and models allows the early production of prototypes and test cases. Planning to create evolutionary prototypes reduces the time lost repeating the work needed to create throwaway prototypes. Making use of paper prototypes early in the design process can also help to avoid programming products that do not satisfy customer expectations. Getting the architectural design right before beginning actual development is also important to avoiding schedule slippage and cost overruns.

Planning is important but should be done expeditiously to avoid delaying the start of development. Developers should have a general idea about how long a project will take to complete, but they need to recognize that they are not likely to know all the project requirements until the software products are delivered. Developers would be wise to avoid detailed planning that extends beyond planning the current prototype. The developers and stakeholders should adopt a process for adding features to be implemented in future prototypes and to assess the impact of these changes on the project schedule and budget.

Risk assessment and acceptance testing are an important part of the prototype assessment process. Having an agile philosophy about managing requirements and adding new features to the final product is important as well. The biggest challenges developers have with evolutionary process models is managing scope creep while delivering a product that meets customer expectations and doing all this while delivering the product on time and within budget. That's what makes software engineering so challenging and rewarding.

## Problems and Points to Ponder

**4.1.** How does the Extreme Programming (XP) model differ from the spiral model in its treatment of incremental prototypes?

**4.2.** Write the acceptance criteria for the user story that describe the use of the "favorite places" or "favorites" feature found on most Web browsers that you wrote for Problem 3.7 in Chapter 3.

**4.3.** How would you create a preliminary architectural design for the first prototype for a mobile app that lets you create and save a shopping list on your device?

**4.4.** Where would you get the historic date needed to estimate the development time for the user stories in a prototype before it is written?

**4.5.** Create a series of sketches representing the key screens for a paper prototype for the shopping list app you created in Problem 4.3.

**4.6.** How can you test the viability of the paper prototype you created for Problem 4.5?

**4.7.** What data points are needed to make the go, no-go decision during the assessment of an evolutionary prototype?

**4.8.** What is the difference between reactive and proactive maintenance?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

# 5 HUMAN ASPECTS OF SOFTWARE ENGINEERING

In a special issue of *IEEE Software*, the guest editors [deS09] make the following observation:

> Software engineering has an abundance of techniques, tools, and methods designed to improve both the software development process and the final product. However, software isn't simply a product of the appropriate technical solutions applied in appropriate technical ways. Software is developed by people, used by people, and supports interaction among people. As such, human characteristics, behavior, and cooperation are central to practical software development.
>
> Without a team of skilled and motivated people, success is unlikely.

**KEY CONCEPTS**


agile team ..... 78
global teams ..... 80
jelled team ..... 76
psychology, software engineering ..... 75
social media ..... 79
team attributes ..... 77
team structures ..... 78
team toxicity ..... 77
traits, software engineer ..... 75


## QUICK LOOK

**What is it?** At the end of the day, *people* build computer software. The human aspects of software engineering often have as much to do with the success of a project as the latest and greatest technology.

**Who does it?** Individuals and teams do software engineering work. In some cases, one person has much of the responsibility, but in most industry-grade software efforts, a team of people does the work.

**Why is it important?** A software team will be successful only if the dynamics of the team are right. It is essential for software engineers on a team to play well with their colleagues and with other product stakeholders.

**What are the steps?** First, you need to try to emulate personal characteristics of successful software engineers. Next, you should appreciate the complex psychology of software engineering work so that you can navigate your way through a project without peril. Then, you need to understand the structure and dynamics of software teams. Finally, you should appreciate the impact of social media, the cloud, and other collaborative tools.

**What is the work product?** Better insight into the people, the process, and the product.

**How do I ensure that I've done it right?** Spend the time to observe how successful software engineers do their work, and tune your approach to take advantage of the strengths they project.

## 5.1 CHARACTERISTICS OF A SOFTWARE ENGINEER

So you want to be a software engineer? Obviously, you need to master the technical stuff, learn the skills required to understand the problem, design an effective solution, build the software, and test it in an effort to develop the highest-quality products possible. You need to manage change, communicate with stakeholders, and use appropriate tools as needed. We will discuss these things at great length later in this book.

But there are other things that are equally important—the human aspects that will make you effective as a software engineer. Erasmus [Era09] identifies seven traits that are present when a software engineer exhibits "superprofessional" behavior.

An effective software engineer has a sense of *individual responsibility*. This implies a drive to deliver on her promises to peers, stakeholders, and her management. It implies that she will do what needs to be done, when it needs to be done in an overriding effort to achieve a successful outcome.

An effective software engineer has an *acute awareness* of the needs of other team members, the stakeholders requesting changes to an existing software solution, and the managers who have overall control of the project. He observes the environment in which people work and adapts his behavior to take both into account.

An effective software engineer is *brutally honest*. If she sees a flawed design, she points out the flaws in a constructive but honest manner. If asked to distort facts about schedules, features, performance, or other product or project characteristics, she opts to be realistic and truthful.

An effective software engineer exhibits *resilience under pressure.* Software engineering is always on the edge of chaos. Pressure comes in many forms—changes in requirements and priorities, demanding stakeholders, and overbearing managers. An effective software engineer manages pressure so that his performance does not suffer.

An effective software engineer has a *heightened sense of fairness.* She gladly shares credit with her colleagues. She tries to avoid conflicts of interest and never acts to sabotage the work of others.

An effective software engineer exhibits *attention to detail.* This does not imply an obsession with perfection. He carefully considers the broader criteria (e.g., performance, cost, quality) that have been established for the product and the project in making his daily technical decisions.

Finally, an effective software engineer is pragmatic. She recognizes that software engineering is not a religion in which dogmatic rules must be followed, but rather a discipline that can be adapted based on the circumstances at hand.

## 5.2 THE PSYCHOLOGY OF SOFTWARE ENGINEERING

In a seminal paper on the psychology of software engineering, Bill Curtis and Diane Walz [Cur90] suggest a layered behavioral model for software development (Figure 5.1). At an individual level, software engineering psychology focuses on recognition of the problem to be solved, the problem-solving skills required to solve it, and the motivation to complete the solution within the constraints established by outer layers in the model. At the team and project levels, group dynamics becomes the